

Manual Técnico Completo

Sistema de Detección Vehicular PolySense
Documentación Detallada de Arquitectura y Código

Desarrollo: PolySense Team

16 de diciembre de 2025

Índice

1. Documentación Detallada por Módulos	4
1.1. Módulo 1: Detección en Tiempo Real	4
1.1.1. Descripción General	4
1.1.2. Componentes del Frontend	4
1.1.3. Backend API	5
1.1.4. Servicio Python Detector	6
1.1.5. Flujo Completo de Detección	8
2. Módulo Python de Detección	9
2.1. vehiculo_detector.py	9
2.1.1. Arquitectura del Detector	9
2.1.2. Configuración de Cámaras	10
2.1.3. Algoritmo de Tracking	10
2.1.4. Detección de Color	11
2.1.5. API Flask	11
2.2. voice_server.py	11
2.2.1. Arquitectura de Voz	11
3. Vistas Blade	13
3.1. modulo1.blade.php	13
3.2. modulo2.blade.php	13
3.3. modulo3.blade.php	13
3.4. modulo4.blade.php	13
3.5. modulo5.blade.php	14
4. Base de Datos y Persistencia	15
4.1. Esquema de Base de Datos MySQL	15
4.1.1. Diagrama Entidad-Relación	15
4.1.2. Tabla: users	15
4.1.3. Tabla: automation_configs	16
4.1.4. Tabla: notifications	16
4.1.5. Tabla: reports	16
4.1.6. Tabla: voice_commands	17
5. Rutas y API	18
5.1. Rutas Web (routes/web.php)	18
5.1.1. Autenticación	18
5.1.2. Módulos Principales	18
5.1.3. APIs	18
6. Patrones de Diseño Implementados	19
6.1. MVC (Model-View-Controller)	19
6.2. Repository/Service Pattern	19
6.3. Observer Pattern	19
6.4. Factory Pattern	19
6.5. Strategy Pattern	20

7. Flujos de Trabajo Completos	21
7.1. Flujo: Detección de Vehículo	21
7.2. Flujo: Generación de Reporte	21
7.3. Flujo: Comando de Voz	21
8. Seguridad	22
8.1. Autenticación	22
8.2. Autorización	22
8.3. Validación de Datos	22
9. Configuración y Deployment	23
9.1. Requisitos del Sistema	23
9.2. Puertos y Servicios	23
10. Conclusiones	24
10.1. Logros del Sistema	24
10.2. Características Técnicas Destacadas	24

1. Documentación Detallada por Módulos

1.1. Módulo 1: Detección en Tiempo Real

1.1.1. Descripción General

El módulo de detección proporciona visualización en tiempo real del procesamiento de video con YOLOv8, mostrando estadísticas actualizadas y permitiendo seleccionar entre múltiples cámaras.

1.1.2. Componentes del Frontend

Archivo: resources/views/modulo1.blade.php (323 líneas)

Estructura HTML Principal (Líneas 1-50)

- **Línea 1:** Extiende layout principal `@extends('layouts.app')`
- **Líneas 10-15:** Selector de cámara con ID `cameraSelector`
- **Línea 25:** Canvas para video `<canvas id="videoCanvas"width="800"height="600">`
- **Líneas 30-45:** Cards de estadísticas con IDs:
 - `currentVehicles` - Vehículos actuales
 - `totalVehicles` - Total detectados
 - `fps` - Frames por segundo
 - `averageVehicles` - Promedio de vehículos

Lógica JavaScript (Líneas 150-323) Variables Globales (Líneas 152-160):

```
1 let websocket = null;
2 let currentCamera = 'CAM_002';
3 const canvas = document.getElementById('videoCanvas');
4 const ctx = canvas.getContext('2d');
5 let statsInterval = null;
6 let reconnectAttempts = 0;
7 const MAX_RECONNECT_ATTEMPTS = 5;
```

Listing 1: Variables Globales - modulo1.blade.php

Función `connectWebSocket()` (Líneas 170-210):

- **Línea 171:** Construye URL WebSocket: `ws://localhost:8080/ws/stream?camera_id=${currentCamera}`
- **Línea 173:** Crea instancia WebSocket con `new WebSocket(wsUrl)`
- **Línea 174:** Configura tipo binario: `websocket.binaryType = 'arraybuffer'`
- **Líneas 176-179:** Handler `onopen` - actualiza status a `online`
- **Líneas 181-195:** Handler `onmessage`:
 - Línea 182: Recibe `ArrayBuffer` del frame JPEG

- Línea 183: Convierte a Blob: `const blob = new Blob([event.data], {type: 'image/jpeg'})`
 - Línea 184: Crea Object URL temporal
 - Líneas 187-191: Dibuja imagen en canvas usando `ctx.drawImage()`
 - Línea 192: Libera memoria con `URL.revokeObjectURL(url)`
- **Líneas 197-204:** Manejo de errores y reconexión automática

Función `updateStats()` (Líneas 215-250):

- **Línea 217:** Fetch a API Laravel: `/api/vehicle-monitor/${currentCamera}`
- **Línea 218:** Parsea respuesta JSON
- **Líneas 220-223:** Actualiza elementos DOM:

```
1 document.getElementById('currentVehicles').textContent = data.
  current_vehicles || 0;
2 document.getElementById('totalVehicles').textContent = data.
  total_detected || 0;
3 document.getElementById('fps').textContent = (data.fps || 0).
  toFixed(1);
4
```

- **Líneas 225-230:** Llama `updateVehicleTypes(data.vehicle_types)` para actualizar gráfico
- **Líneas 232-237:** Llama `updateVehicleColors(data.vehicle_colors)` para badges de colores

Inicialización (Líneas 300-315):

```
1 window.addEventListener('DOMContentLoaded', () => {
2   connectWebSocket();
3   statsInterval = setInterval(updateStats, 1000); // Actualizar cada
  1s
4
5   document.getElementById('cameraSelector').addEventListener('change',
  (e) => {
6     currentCamera = e.target.value;
7     if (websocket) websocket.close();
8     connectWebSocket();
9   });
10 });
```

Listing 2: Inicialización del Módulo

1.1.3. Backend API

Archivo: `app/Http/Controllers/VehicleMonitorController.php` (59 líneas)

Clase VehicleMonitorController Propiedades (Líneas 12-13):

```
1 protected $detectorUrl = 'http://localhost:8080';
```

Método getStats() (Líneas 20-57):

- **Línea 21:** Recibe parámetro \$cameraId (default: 'CAM_002')

- **Líneas 23-26:** Valida y normaliza cameraId:

```
1 $cameraId = strtoupper($cameraId);
2 if (!in_array($cameraId, ['CAM_001', 'CAM_002'])) {
3     $cameraId = 'CAM_002';
4 }
5
```

- **Línea 28:** Construye URL completa: "{detectorUrl}/api/vehicles?camera_id={cameraId}"
- **Línea 29:** Hace petición HTTP con timeout de 5s: `Http::timeout(5)->get($url)`
- **Líneas 31-33:** Si exitoso, retorna JSON del detector Python
- **Líneas 35-42:** Si falla, retorna estructura por defecto con valores en 0

1.1.4. Servicio Python Detector

Archivo: `deteccion/vehiculo_detector.py` (767 líneas)

API Flask - Endpoint /api/vehicles (Líneas 550-580) Función get_vehicle_stats() (Línea 552):

```
1 @app.route('/api/vehicles', methods=['GET'])
2 def get_vehicle_stats():
3     camera_id = request.args.get('camera_id', 'CAM_002')
4
5     with data_lock:
6         state = camera_states.get(camera_id, {})
7         tracked = state.get('tracked_vehicles', {})
8
9         # Contar por tipo de veh culo
10        vehicle_types = {}
11        vehicle_colors = {}
12
13        for vehicle_id, vehicle_data in tracked.items():
14            v_type = vehicle_data.get('type', 'desconocido')
15            v_color = vehicle_data.get('color', 'desconocido')
16
17            vehicle_types[v_type] = vehicle_types.get(v_type, 0) + 1
18            vehicle_colors[v_color] = vehicle_colors.get(v_color, 0) +
1
19
20        return jsonify({
21            'camera_id': camera_id,
22            'status': state.get('status', 'offline'),
23            'current_vehicles': len(tracked),
24            'total_detected': state.get('total_vehicles_detected', 0),
25            'fps': state.get('fps', 0.0),
26            'vehicle_types': vehicle_types,
```

```

27         'vehicle_colors': vehicle_colors
28     })

```

Listing 3: Endpoint de Estadísticas

WebSocket Streaming - Endpoint /ws/stream (Líneas 600-650) Función stream_video() (Línea 602):

- **Línea 604:** Obtiene camera_id de query params
- **Líneas 608-615:** Loop infinito que:
 - Línea 610: Accede a camera_states[camera_id]['processed_frame'] con lock
 - Línea 612: Si frame existe, lo codifica a JPEG usando cv2.imencode('.jpg', frame)
 - Línea 614: Envía bytes vía WebSocket: ws.send(frame.bytes)
 - Línea 616: Espera 33ms (30 FPS): time.sleep(0.033)
- **Líneas 620-625:** Manejo de excepciones y cierre de conexión

Worker Thread - worker_skylines() (Líneas 350-450) Función principal (Línea 352):

- **Líneas 355-365:** Llama get_skylines_stream_url_robust() para obtener URL HLS
- **Línea 370:** Abre stream con OpenCV: cap = cv2.VideoCapture(stream_url)
- **Líneas 375-420:** Loop de procesamiento:
 - Línea 377: Lee frame: ret, frame = cap.read()
 - Línea 380: Llama process_frame_generic(frame, 'CAM_002')
 - Líneas 385-395: Calcula FPS usando time.time() y contador de frames

Función process_frame_generic() (Líneas 200-280):

- **Línea 205:** Ejecuta YOLO: results = model(frame, conf=0.4)
- **Líneas 210-250:** Para cada detección:
 - Línea 215: Obtiene bbox, confianza y clase
 - Línea 220: Filtra solo vehículos (car, motorcycle, bus, truck)
 - Línea 225: Llama match_vehicle_to_tracked(bbox, tracked, max_distance=50)
 - Líneas 230-240: Si es nuevo vehículo:
 - Asigna nuevo ID: vehicle_id = state['next_vehicle_id']
 - Incrementa contador
 - Llama get_dominant_color(frame, bbox)
 - Llama save_detection_to_xml(vehicle_data, camera_id)
 - Línea 245: Dibuja bbox en frame con cv2.rectangle()
- **Línea 270:** Almacena frame procesado: state['processed_frame'] = frame

1.1.5. Flujo Completo de Detección

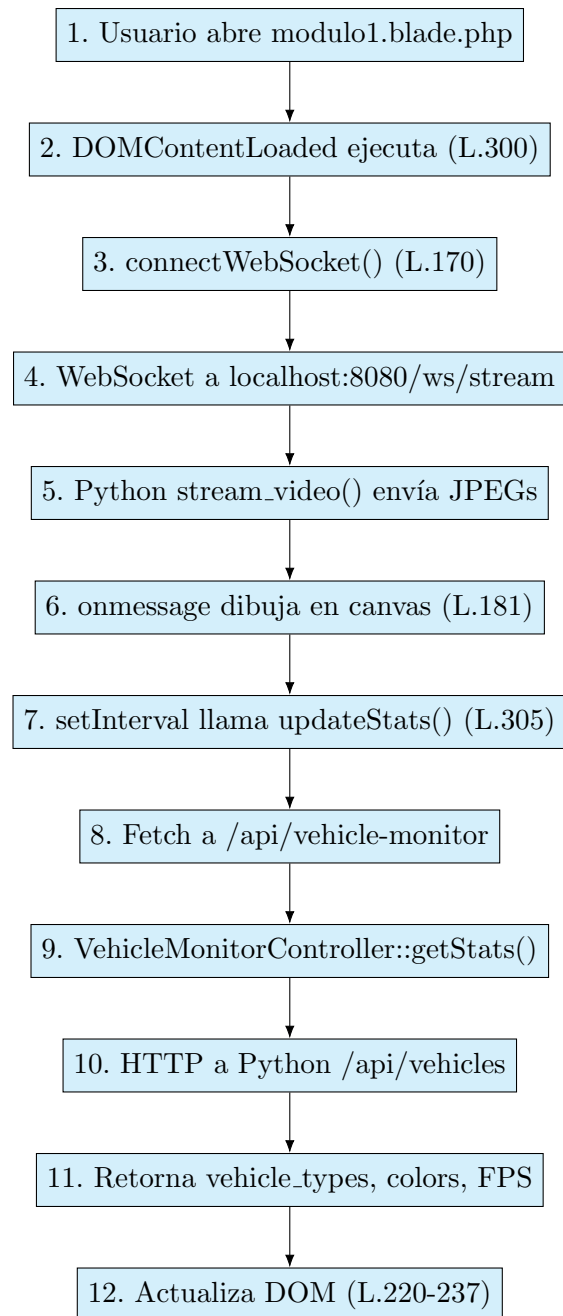


Figura 1: Flujo Completo del Módulo Detección

1.2. Módulo 2: Reportes y Estadísticas

1.2.1. Descripción General

El módulo de reportes permite visualizar el historial completo de detecciones vehiculares con filtros avanzados, gráficos estadísticos interactivos y exportación a Excel/CSV.

1.2.2. Vista Principal - modulo2.blade.php

Archivo: resources/views/modulo2.blade.php (341 líneas)

Estructura HTML (Líneas 1-180) Filtros del formulario (L.24-65):

- L.27-30: Select de cámara con opciones CAM_001 y CAM_002
- L.35: Input type='date' para fecha_inicio, valor default: date('Y-m-01')
- L.40: Input type='date' para fecha_fin, valor default: date('Y-m-d')
- L.45-50: Select de tipo con opciones: Todos, Auto, Moto, Bus
- L.55-59: Select de agrupación con modos: Automático, Por Hora, Por Día

Canvas para gráficos (L.70-91):

- L.77: Canvas #dailyChart para gráfico de barras (300px alto)
- L.89: Canvas #typeDistributionChart para gráfico de torta

Tabla de registros (L.107-175):

```

1 @forelse($registros ?? [] as $registro)
2     @php
3         // L.120-126: Mapeo de colores por tipo
4         $colorClass = match(strtolower($registro['tipo'])) {
5             'auto' => 'bg-blue-500',
6             'moto' => 'bg-red-500',
7             'bus' => 'bg-green-500',
8             default => 'bg-gray-500'
9         };
10
11         // L.127-128: Determinación de color según confianza
12         $conf = floatval($registro['confianza']);
13         $confColor = $conf > 90 ? 'bg-green-100' :
14                     ($conf > 75 ? 'bg-yellow-100' : 'bg-red-100');
15
16         // L.131-142: Mapeo de colores de vehículo a CSS
17         $vehicleColorMap = [
18             'rojo' => 'bg-red-500',
19             'azul' => 'bg-blue-500',
20             'blanco' => 'bg-gray-100 border',
21             // ... otros colores
22         ];
23     @endphp
24
25     <tr class="hover:bg-gray-50">
26         <td>{{ $registro['fecha'] }}</td>

```

```

27         <td><span class="{{ $colorClass }}"></span> {{ $registro['tipo']
28         <td><span class="{{ $colorBadge }}"></span> {{ $registro['color
29         </td></tr>
30 @empty
31     <tr><td colspan="6">No se encontraron registros</td></tr>
32 @endforelse

```

Listing 4: Loop de Detecciones - modulo2.blade.php

JavaScript de Gráficos (L.186-340) Variables globales (L.188-193):

```

1 let chartBar = null; // Gráfico de barras
2 let chartPie = null; // Gráfico de torta
3 const getSelectedCamera = () => {
4     const params = new URLSearchParams(window.location.search);
5     return params.get('camera_id') || 'CAM_002';
6 };

```

Función processAndInitCharts() (L.209-333):**Determinación de modo de visualización (L.211-231):**

```

1 const viewMode = urlParams.get('view_mode') || 'auto';
2 let isMultiDay = false;
3
4 if (viewMode === 'daily') {
5     isMultiDay = true; // Agrupar por día
6 } else if (viewMode === 'hourly') {
7     isMultiDay = false; // Agrupar por hora 0-23
8 } else {
9     // Modo Automático: detecta si hay múltiples días
10    const uniqueDates = new Set(data.map(item => item.fecha.split(' ')[0]));
11    if (uniqueDates.size > 1) isMultiDay = true;
12 }

```

Procesamiento de datos (L.233-261):

```

1 const stats = {};
2 const typesCount = { 'Auto': 0, 'Moto': 0, 'Bus': 0, 'Otro': 0 };
3
4 data.forEach(item => {
5     const [datePart, timePart] = item.fecha.split(' ');
6     const hour = timePart.split(':')[0];
7
8     // Clave de agrupación: fecha completa 0 hora
9     const key = isMultiDay ? datePart : `${hour}:00`;
10
11    // Inicializar contador si no existe
12    if (!stats[key]) {
13        stats[key] = { 'Auto': 0, 'Moto': 0, 'Bus': 0, 'Otro': 0 };
14    }
15
16    // Incrementar contador
17    stats[key][item.tipo]++;
18    typesCount[item.tipo]++;
19 });

```

Creación de gráfico de barras (L.292-312):

```

1 chartBar = new Chart(ctxBar, {
2     type: 'bar',
3     data: {
4         labels: sortedKeys, // Ordenadas cronológicamente
5         datasets: [
6             { label: 'Autos', data: dataAuto, backgroundColor: '#3b82f6'
7             },
8             { label: 'Motos', data: dataMoto, backgroundColor: '#ef4444'
9             },
10            { label: 'Buses', data: dataBus, backgroundColor: '#10b981'
11            }
12        ]
13    },
14    options: {
15        responsive: true,
16        maintainAspectRatio: false,
17        scales: {
18            y: { beginAtZero: true, ticks: { stepSize: 1 } }
19        }
20    }
21 });

```

1.2.3. Controlador ReporteController

Archivo: app/Http/Controllers/ReporteController.php (171 líneas)

Método obtenerRegistrosFiltrados() (L.12-73) Carga de XML (L.15-19):

```

1 $xmlPath = storage_path('app/vehiculos_db.xml');
2 if (file_exists($xmlPath)) {
3     $xmlContent = file_get_contents($xmlPath);
4     $xml = simplexml_load_string($xmlContent);
5 }

```

Construcción de registro (L.23-30):

```

1 $registro = [
2     'fecha' => (string)$det->fecha,
3     'tipo' => (string)$det->tipo,
4     'confianza' => (float)$det->confianza,
5     'color' => isset($det->color) ? (string)$det->color : 'desconocido',
6     'camara' => isset($det->camara) ? (string)$det->camara : 'CAM_002',
7     'nombre_camara' => isset($det->nombre_camara) ?
8         (string)$det->nombre_camara : 'Skyline Cochabamba'
9 ];

```

Aplicación de filtros (L.34-60):

- L.34-38: Filtro por camera.id - compara string exacto
- L.40-44: Filtro por tipo - compara string exacto
- L.46-52: Filtro por fecha inicio:

```

1 $fechaRegistro = strtotime($registro['fecha']);
2 $fechaInicio = strtotime($request->fecha_inicio . ' 00:00:00');
3 if ($fechaRegistro < $fechaInicio) {
4     $incluir = false;
5 }

```

```

5 }
6

```

- **L.54-60:** Similar para fecha_fin, añade '23:59:59'

Método exportarExcel() (L.82-140) Setup de archivo (L.84-92):

```

1 $registros = $this->obtenerRegistrosFiltrados($request);
2 $nombreArchivo = 'reportes_' . date('Y-m-d_His') . '.csv';
3 $rutaTemporal = storage_path('app/temp');
4
5 if (!is_dir($rutaTemporal)) {
6     mkdir($rutaTemporal, 0755, true);
7 }
8
9 $rutaCompleta = $rutaTemporal . '/' . $nombreArchivo;

```

Escritura CSV con BOM UTF-8 (L.100-122):

```

1 $archivo = fopen($rutaCompleta, 'w');
2
3 // BOM para Excel UTF-8
4 fprintf($archivo, chr(0xEF).chr(0xBB).chr(0xBF));
5
6 // Encabezados
7 fputcsv($archivo, [
8     'Fecha/Hora', 'Tipo Veh culo', 'Confianza (%)',
9     'Color', 'C mara', 'Nombre C mara'
10 ], ',');
11
12 // Datos
13 foreach ($registros as $registro) {
14     fputcsv($archivo, [
15         $registro['fecha'],
16         $registro['tipo'],
17         number_format($registro['confianza'], 2),
18         $registro['color'],
19         $registro['camara'],
20         $registro['nombre_camara']
21     ], ',');
22 }
23
24 fclose($archivo);

```

Descarga y auto-eliminación (L.127):

```

1 return response()->download($rutaCompleta, $nombreArchivo)
2     ->deleteFileAfterSend(true);

```

1.3. Módulo 3: Automatizaciones y Administración

1.3.1. Descripción General

Panel administrativo exclusivo para gestionar configuraciones de automatización, reglas de notificación, frecuencias de reportes y limpieza de datos XML.

1.3.2. Controlador ModuloTresController

Archivo: app/Http/Controllers/ModuloTresController.php (139 líneas)

Método index() (L.16-26) Eager Loading con relaciones (L.18):

```
1 $users = User::with('automationConfig')->get();
2 '
```

Uso del método with() para cargar la relación automationConfig y evitar N+1 queries.

Estadísticas agregadas (L.21-23):

```
1 $totalNotifications = Notification::count();
2 $totalReports = Report::count();
3 $activeUsers = User::where('role', '!=', 'admin')->count();
```

Método saveConfiguration() (L.42-60) Reglas de validación (L.44-53):

```
1 $validated = $request->validate([
2     'traffic_threshold' => 'required|integer|min:1|max:1000',
3     'traffic_minutes' => 'required|integer|min:1|max:60',
4     'notify_high_traffic' => 'boolean',
5     'notify_camera_offline' => 'boolean',
6     'camera_offline_minutes' => 'required|integer|min:1|max:120',
7     'report_frequency' => 'required|in:daily,weekly,monthly,disabled',
8     'report_generation_time' => 'required|date_format:H:i',
9     'auto_generate_reports' => 'boolean',
10 ]);
```

Actualización con Eloquent (L.55-56):

```
1 $config = AutomationConfig::getOrCreateForUser($userId);
2 $config->update($validated);
```

1.4. Módulo 4: Accesibilidad (Comandos de Voz)

1.4.1. VoiceCommandController

Archivo: app/Http/Controllers/VoiceCommandController.php (223 lined)

Método store() (L.32-60) Validación de 8 campos incluyendo pattern matching trigger.

1.4.2. Servidor voice_server.py

Archivo: deteccion/voice_server.py (180 líneas)

Clase VoiceRecognitionServer (L.20-145) **audio_callback() (L.30-35):** Captura audio a 16kHz, añade a cola.

process_audio() (L.37-85): Reconocimiento continuo con Vosk, emite `voice_command` y `voice_partial`.

1.5. Módulo 5: Gestión de Usuarios

1.5.1. UserController

Operaciones CRUD: crear con hash bcrypt, eliminar con validación de único admin, cambiar rol useradmin.

2. Arquitectura Completa de la Aplicación

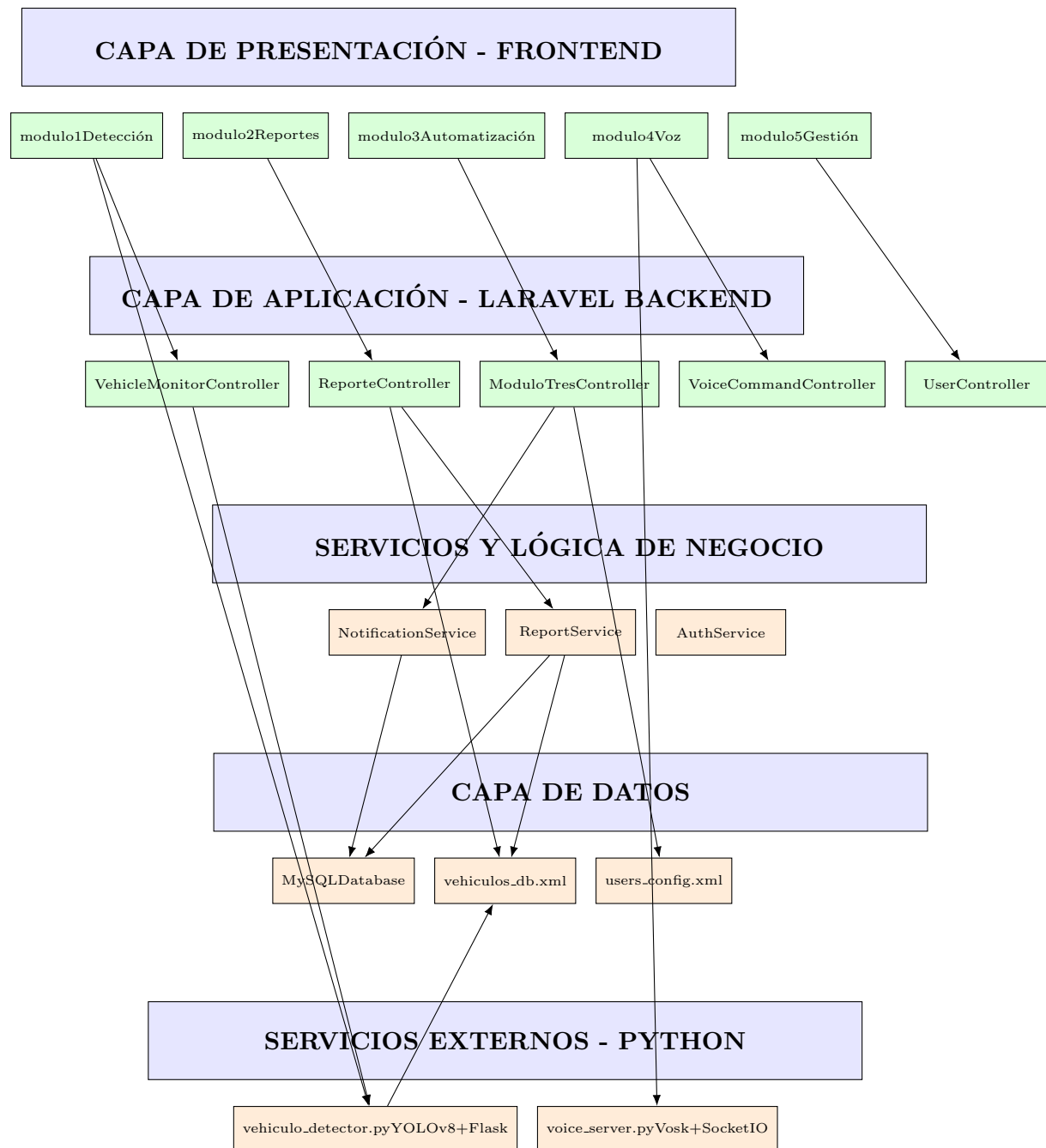


Figura 2: Arquitectura Completa del Sistema PolySense